

Fibonacci-szeru szalkezeles kodmagyarazat (pthread)

Ez a program parhuzamosan szamolja ki, hogy adott Fibonacci-szeru sorozatok hany lepesben kozelitik meg az aranymetszes erteket (1.618...), egy adott epsilon pontossaggal.

1. A ``fibonacci_steps()`` fuggveny meghatarozza, hogy adott (a, b) kezdoparbol indulva hany lepes kell az aranymetszes kozeliteséhez.
2. A ``worker()`` fuggveny egy szalban fut, meghivja a ``fibonacci_steps()``-et, majd frissiti a globalis minimum lepesszamot, ha szukseges.
3. A ``run_simulation()`` kezeli a szalak létrehozast, adatok inicializalasat, es futasidot is mer.
4. Az egesz szamitas parhuzamositva van ``pthread`` szalakkal, mutex vedelemmel.
5. Vegul az ``main()`` tobb kulonbozo epsilon ertekkel hivja meg a szimulaciot, hogy demonstralja az idobeli kulonbsegeket.

Reszletes pthread fuggveny magyarázat

1. pthread_t

- A szál azonosító típusa.
- Minden szálhoz tartozik egy pthread_t változó.

2. pthread_create

- Új szál létrehozása.
- Paraméterei: szál azonosító, attribútum (általában NULL), futtatandó függvény, argumentum.

3. pthread_join

- Megvárja, amíg egy adott szál befejeződik.
- Szinkronizációs célból fontos.

4. pthread_mutex_t

- Egy mutex objektum típusa.
- Kizárólagos hozzáférést biztosít kritikus szakaszhoz.

5. pthread_mutex_lock

- Belepes a kritikus szakaszba, zárja a mutexet.
- Ha már másik szál zárta, akkor vár.

6. pthread_mutex_unlock

- Kilepes a kritikus szakaszból, feloldja a mutexet.
- Ezután más szál is belephet.

7. Kritikus szakasz védelem

- A `global_min_steps` változó módosítása mutex védelemmel történik.
- Ezzel versenyhelyzetet (race condition) kerülünk el.

Ezek a függvények a POSIX szabványnak megfelelően működnek és elengedhetetlenek a biztonságos, párhuzamos programozásban.